AI-Powered Job Search & Application Tracker

Full-stack project plan • MVP scope • architecture • AI provider failover • implementation roadmap

Project goal

Build a personal job-search platform that accepts a résumé, discovers relevant job opportunities, scores each role against the candidate’s actual experience, helps tailor an application résumé without inventing qualifications, and tracks recruiter emails through Gmail.

Recommended approach: build a focused MVP first. Do not attempt to crawl the entire internet in version one. Start with a small number of reliable job sources and a clear, explainable matching workflow; add more sources and automation after the core system works.

Prepared: October 30, 2026

1. Product overview

The product is a personal job-search command center. It combines job discovery, résumé analysis, job-fit scoring, résumé tailoring, application tracking, and email monitoring in one dashboard.

Primary user journey

- Upload a PDF or DOCX résumé and extract its text and structured details.
- Review a résumé profile: skills, technologies, work history, projects, education, and experience level.
- Discover jobs from configured sources and normalize them into a common job record.
- Compare each job description with the résumé and assign a fit category with evidence, strengths, and gaps.
- Open a job, review the match explanation, and optionally generate a tailored résumé draft.
- Review and edit the draft before downloading or using it in an application.
- Connect Gmail to detect relevant recruiter and employer messages and update application timelines.

2. Core modules and requirements

A. Authentication and candidate profile

- Register/sign in, manage account settings, and protect personal résumé and email data.
- Store one active master résumé initially; later support multiple résumé versions.
- Allow the user to correct extracted skills, dates, titles, and project details.

B. Résumé upload and parsing

- Accept PDF and DOCX; validate file type and size, store securely, and extract text.
- Parse structured sections: summary, skills, experience, projects, education, certifications, and links.
- Keep the original file unchanged. Store extracted text and structured data separately.
- Show a review screen because parsing can misread columns, dates, tables, and unusual layouts.

C. Job discovery

- Use source adapters so each job board or company career page has its own ingestion implementation.
- Start with one or two permitted sources or feeds. Prefer official APIs, RSS feeds, public career pages, and sources whose terms allow collection.
- Avoid indiscriminate crawling, bypassing CAPTCHAs, or ignoring robots.txt and site terms. Respect rate limits and attribution requirements.
- Normalize title, company, location, remote status, employment type, salary when available, description, source URL, posted date, and discovered date.
- Deduplicate using source job ID where available, otherwise a normalized company/title/location/URL fingerprint.

D. Job-fit scoring and categorization

Use three user-facing categories:

Category	Suggested rule	Meaning
Strong fit	80–100	Core requirements align; only minor gaps or preferences are missing.
Relevant	55–79	Several skills align, but there are meaningful gaps or experience-level differences.
Low match	0–54	Major requirements are missing or the role is outside the candidate's target.

- Treat these score bands as initial product defaults, not objective truth. Tune them after reviewing real matches.

- Show a score plus reasons: matching skills, relevant experience, missing must-have requirements, seniority mismatch, location/work arrangement, and other preferences.
- Separate must-have requirements from nice-to-have requirements. A missing mandatory license or language should matter more than a missing optional framework.
- Never claim a skill or qualification exists unless it appears in the résumé or the user confirms it.

E. Dashboard and job tracker

- Table columns: fit category, score, job title, company, location/remote, date posted, date found, and application status.
- Filters: category, role keywords, location, remote, source, date, and status; sorting by fit score and recency.
- Statuses: Saved, To review, Tailoring, Ready to apply, Applied, Interview, Assessment, Offer, Rejected, Withdrawn, Archived.
- Job detail page: full description, source link, fit explanation, strengths, gaps, notes, résumé version, and activity history.

F. Résumé tailoring

- Generate a job-specific draft using the master résumé and the selected job description as source material.
- Reorder and emphasize existing relevant experience; improve phrasing and keyword alignment without fabricating tools, metrics, responsibilities, or outcomes.
- Flag any suggested claim that needs user confirmation. Do not silently add unsupported claims.
- Provide side-by-side original and tailored versions, editable text, and PDF/DOCX export as a later enhancement.
- Require human review and explicit approval before the user sends an application.

G. Gmail integration and email tracking

- Use Google OAuth with the minimum Gmail scopes needed. Start with read-only access; request sending permission only if a send-email feature is deliberately added.
- Detect likely application-related messages using sender, subject, body, company, job title, and known application records.
- Show matched messages in an Inbox/Updates page and link them to the relevant job where confidence is sufficient.
- Support manual correction when a message is matched to the wrong job. Avoid automatically changing a status based on uncertain email interpretation.
- Store message IDs and metadata where possible rather than copying more email content than necessary.

3. Recommended technology stack

Layer	Technology	Purpose
Frontend	Next.js + React + TypeScript	Dashboard, routing, server-rendered pages where useful, reusable UI.
UI	Tailwind CSS + shadcn/ui	Responsive layout, accessible components, tables, dialogs, forms.
Backend API	Laravel (PHP)	REST API, business logic, validation, auth, integrations, provider orchestration.
Authentication	Laravel Sanctum	Session or token-based authentication between Next.js and Laravel.
Database	PostgreSQL	Users, résumé data, jobs, scores, applications, email references, audit records.
Queue/cache	Redis + Laravel Queues	Background parsing, job ingestion, AI calls, email sync, retry and cooldown state.
File storage	Local in development; S3-compatible storage in production	Original résumé and generated exports with private access.
AI providers	Gemini, Groq, OpenRouter (availability and limits vary)	Résumé extraction assistance, structured matching, explanations, tailoring.
Email	Gmail API + Google OAuth	Read and classify relevant job-application messages.
Testing	PHPUnit/Pest, Laravel HTTP tests, Vitest, Playwright	Unit, API, UI, and end-to-end coverage.
Deployment	Vercel (Next.js) + Laravel-compatible host + managed Postgres/Redis	Deploy frontend and API separately; use environment-specific secrets.

Architecture recommendation: keep Laravel as the single source of business logic and integration orchestration. Next.js should call Laravel endpoints rather than calling AI providers directly from the browser. Keep provider keys, Gmail credentials, and storage secrets on the server.

4. AI provider strategy and automatic failover

Yes, automatic provider failover is practical. Implement it as a controlled provider manager in Laravel, rather than scattering provider-specific code throughout the application.

Suggested request flow

- A feature calls one internal service, for example AiGateway, with a task name and structured input.
- The gateway selects an eligible provider based on task support, configured priority, recent failures, and cooldown state.
- If the provider succeeds, validate the response schema and return the result.
- If the provider returns a retryable limit or availability error, record the failure and try the next eligible provider.
- If all providers fail, return a clear temporary-unavailable response and queue a retry where appropriate.

Important safeguards

- Distinguish rate limits, quota exhaustion, timeouts, authentication errors, invalid requests, and server errors. Do not retry every error.
- Use bounded retries, exponential backoff, and jitter. Avoid rapid loops that make provider limits worse.
- Track per-provider cooldowns and quota state in Redis; persist operational metrics for debugging.
- Use task-specific routing. A provider suitable for résumé parsing may not be the best for long-form tailoring.

- Validate structured output (for example, JSON schema) before accepting it. If invalid, allow one controlled repair attempt or move to another provider.
- Set maximum cost/token budgets, request timeouts, and output-length limits even when using free tiers.
- Free-tier quotas, model availability, and terms can change. Never design the system assuming a provider will always offer free access.

Suggested initial priority: configure Gemini, Groq, and OpenRouter as interchangeable adapters, but verify each provider's current free-tier access, model availability, rate limits, and data-use terms before deployment. Make provider order configurable in environment settings or an admin page.

5. High-level architecture

Next.js web app → Laravel API → PostgreSQL. Laravel dispatches background jobs to Redis queues for résumé parsing, job-source ingestion, AI scoring/tailoring, and Gmail synchronization. A Laravel AI gateway routes model requests to provider adapters. Private file storage holds uploaded résumés and generated exports. Gmail OAuth tokens are encrypted at rest.

Keep scraping and AI work asynchronous. The user should see a queued/running/completed/failed state rather than waiting on a long HTTP request.

6. Initial database model

Table	Key fields / purpose
users	Account and authentication identity.
resumes	User, file path, original filename, parsed text, status, active flag, version.
resume_sections	Resume ID, section type, structured content, source text, confidence.
jobs	Source, external ID, title, company, location, description, URL, dates, normalized fields.
job_matches	User, job, score, category, strengths JSON, gaps JSON, explanation, model/version.
applications	User, job, status, notes, applied date, next action, selected résumé version.
resume_versions	User, job/application, content, version number, export path, approval state.
email_connections	User, Google account identifier, encrypted tokens, scopes, sync cursor.
email_messages	Connection, Gmail message ID, metadata, classification, matched application, confidence.
job_sources	Source name, configuration, enabled flag, last run, status, compliance notes.
ai_provider_events	Provider, model, task, latency, outcome, error class, token usage when available.

7. API outline

Example Laravel API routes (final route names can change during implementation):

Method	Route	Purpose
POST	/api/resumes	Upload a résumé and start parsing.
GET	/api/resumes/current	Get the active résumé profile.
PATCH	/api/resumes/current	Correct extracted profile fields.
GET	/api/jobs	List, filter, and sort discovered jobs.
GET	/api/jobs/{id}	Get job details and match explanation.
POST	/api/jobs/{id}/analyze	Request or refresh a fit analysis.
POST	/api/jobs/{id}/tailor	Generate a tailored résumé draft.
PATCH	/api/resume-versions/{id}	Edit and save a draft.
POST	/api/applications	Create or save an application record.
PATCH	/api/applications/{id}	Update status, notes, and next action.
GET	/api/email/messages	List synced and classified email messages.
POST	/api/integrations/google/connect	Begin Google OAuth connection.
POST	/api/integrations/google/sync	Queue an email sync.

8. MVP implementation roadmap

Phase 1 — Foundation

Set up repositories, Next.js app, Laravel API, PostgreSQL, Sanctum, shared environment configuration, and basic dashboard shell.

Phase 2 — Résumé profile

Implement secure upload, PDF/DOCX text extraction, parsing, structured profile storage, and a user review/correction screen.

Phase 3 — Job ingestion

Implement one permitted job source end-to-end, normalization, deduplication, scheduled ingestion, and manual refresh. Add a second source only after the first is stable.

Phase 4 — Matching

Create deterministic baseline scoring for explicit skills and requirements; add AI-generated explanations and gap summaries. Test against real job descriptions.

Phase 5 — Tracker UI

Build the jobs table, filters, job detail page, categories, statuses, notes, and saved jobs.

Phase 6 — Tailoring

Generate evidence-grounded résumé drafts, show changes, allow edits, and export a reviewed version.

Phase 7 — Gmail

Add Google OAuth, minimal read-only Gmail scope, sync, message classification, job matching, and manual correction.

Phase 8 — Reliability

Add provider failover, queue retries, rate-limit handling, monitoring, backups, security review, and deployment hardening.

9. Suggested MVP boundaries

Include in version one: one master résumé, one or two job sources, a job list, three fit categories, clear strengths/gaps, manual status tracking, and a basic tailored résumé draft with human review.

Defer until later: broad web crawling, automatic applications, sending emails, complex multi-user/team features, mobile apps, advanced analytics, browser extensions, and fully automated interview scheduling.

This scope keeps the project achievable while still demonstrating full-stack engineering, background processing, external integrations, AI orchestration, and practical product thinking.

10. Security, privacy, and responsible automation

- Treat résumés, contact details, and email as sensitive personal data. Use HTTPS, authorization checks, encrypted secrets, and private file storage.
- Do not expose AI keys or Google refresh tokens to the browser. Encrypt OAuth tokens at rest and support disconnect/revocation.
- Use least-privilege Google scopes and clearly explain what email data is read and why.
- Provide deletion controls for résumé files, parsed data, job records, and connected account data.
- Respect job-source terms, robots directives, rate limits, and applicable privacy requirements. Prefer official APIs and feeds.
- Require user approval before any application is submitted or any email is sent.
- Log important actions and provider failures without logging full résumé or email contents unnecessarily.

11. Success criteria

- A user can upload a résumé and review an accurate, editable profile.
- The system imports jobs from at least one reliable source and avoids obvious duplicates.
- Every match displays a category, score, strengths, gaps, and a readable explanation.
- A tailored résumé draft remains faithful to verified résumé facts and can be edited before export.
- A user can track application status and associate relevant Gmail messages with a job.
- When one AI provider is rate-limited or unavailable, the gateway can safely try another eligible provider or report a clear failure.
- Background jobs can be retried without creating duplicate jobs, messages, or résumé versions.

12. Key decisions to make before coding

- Which initial job source is permitted and technically practical for the first integration?
- Will the first release support PDF only, or PDF and DOCX?
- Which Google Gmail scope is sufficient for the intended read-only tracking workflow?
- Which provider/model is best for each task after testing free-tier access and output quality?
- What are the initial fit-score weights, and which job preferences should be hard filters?
- Will deployment use separate frontend/API hosting and managed PostgreSQL/Redis?

13. Recommended first development milestone

Build one complete vertical slice before expanding: upload one résumé → parse and review it → ingest a small batch of jobs from one permitted source → score them → display the strong fit/relevant/low match table. This validates the product's central promise before investing in Gmail, advanced tailoring, or many integrations.

Final recommendation

The strongest version of this project is not simply a scraper with an AI score. It is an explainable job-search workflow that helps the user decide which roles deserve attention, prepare accurate applications, and keep every opportunity organized. Build the matching and tracking foundation first; add automation carefully, with clear user control.

Planning note: This document is an implementation blueprint, not a guarantee that third-party APIs or free AI quotas will remain available. Verify current provider pricing, quotas, model access, Gmail API policies, and job-source terms when implementation begins.